

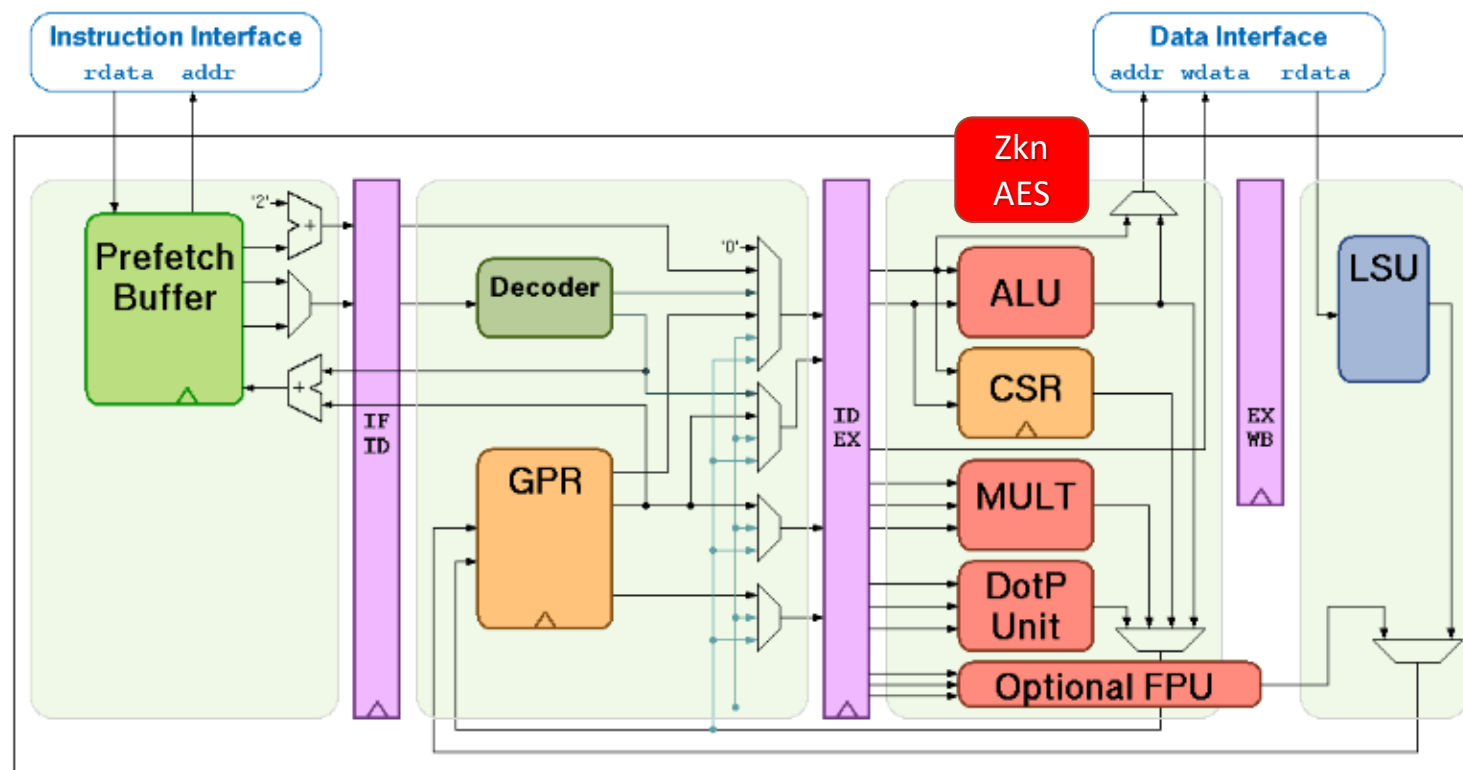
# Processor Design Project

## CESE 4040

## Advanced Encryption Standard

# Assignment

## RI5CY



[RI5CY: User Manual]

```
...  
addi sp, sp, -16  
sd ra, 0(sp)  
sd s1, 8(sp)  
mv s1, a0  
call strlen  
...  
aes32esi t0, t0, a4, 0
```

# Outline

- Encryption
- Advanced Encryption Standard
  - Encryption/Decryption rounds
  - Key scheduling
- RISC-V ISA
- RISC-V Scalar Cryptography Instruction Set Extension
- AES Revisited
  - Encryption/Decryption rounds
  - Key Schedule

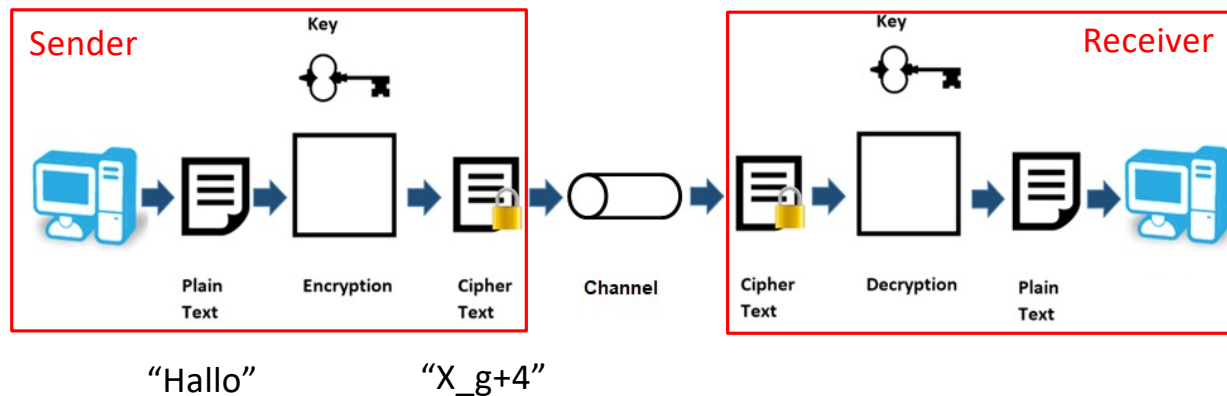
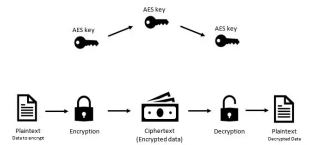
# Encryption (and Decryption)

- Encryption

- The process of encoding a message (plain text) in such a way that its meaning is not obvious anymore

- Decryption

- The reverse process transforms an encrypted message (cipher text) back into the plain text



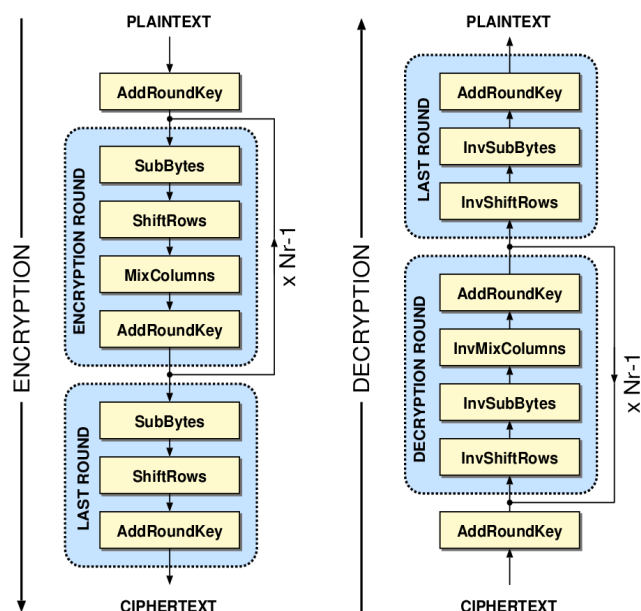
# Advanced Encryption Standard (AES)

## • AES - Advanced Encryption Standard

- Result of National Institute of Standards and Technology (NIST) contest
- Algorithm's name: Rijndael (Vincent Rijmen + John Daemen)
- 128-bit (16 bytes) plaintext often represented by a 4 by 4 byte matrix

Byte 0 (B0)

|    |    |     |     |
|----|----|-----|-----|
| B0 | B4 | B8  | B12 |
| B1 | B5 | B9  | B13 |
| B2 | B6 | B10 | B14 |
| B3 | B7 | B11 | B15 |

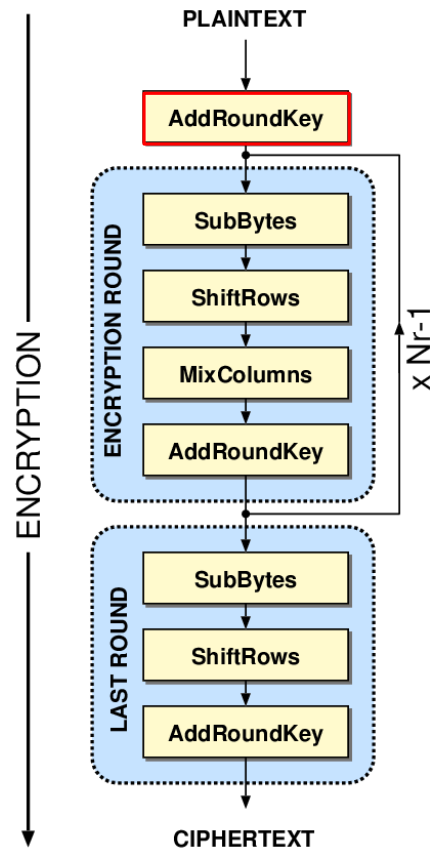


| Key length | # rounds |
|------------|----------|
| 128 bits   | Nr=10    |
| 196 bits   | Nr=12    |
| 256 bits   | Nr=16    |

## Basic Operations:

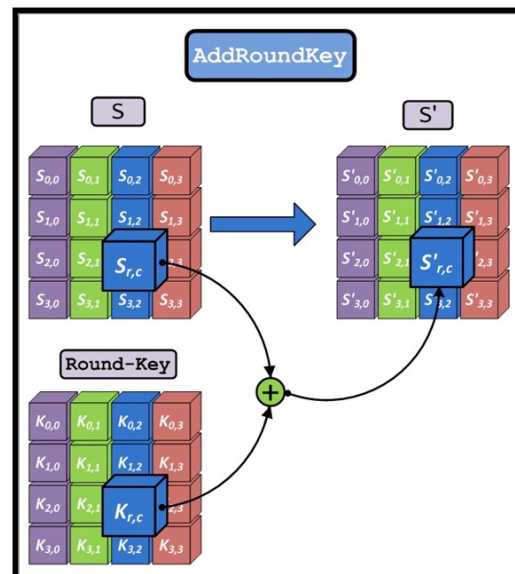
- Substitution Byte – byte level substitutions
- Shift Rows
- Mix Columns
- Add Round Key

# Advanced Encryption Standard – AddRoundKey

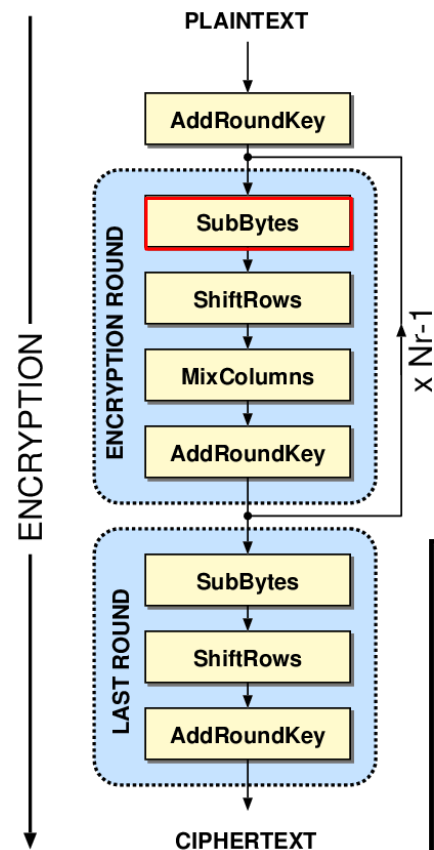


- AddRoundKey

- obscures the relationship between key and plaintext
- Each round uses a different round key which is generated by the Key Schedule
- Consists of XOR operation of the current state with the round key

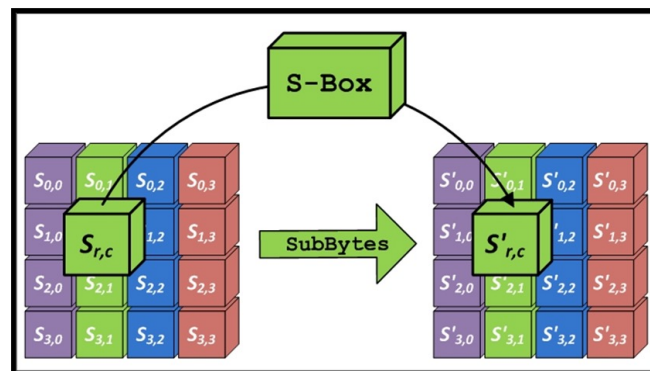


# Advanced Encryption Standard – Substitution



## • Substitution

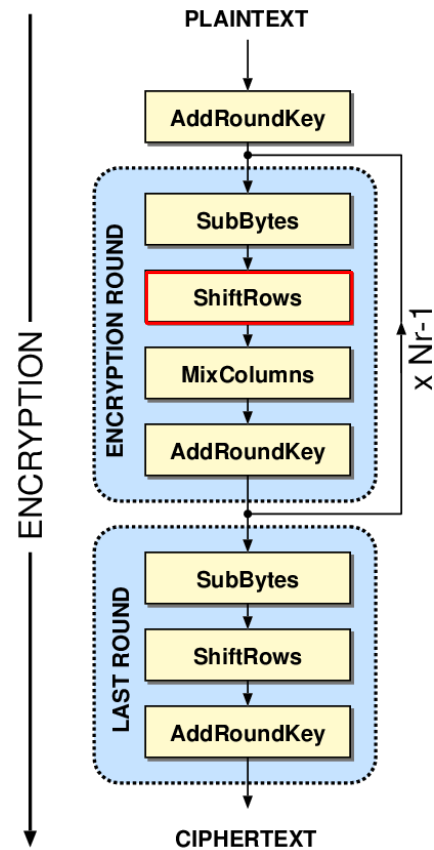
- Obscures the relationship between key and plaintext
- The byte substitution consists of 16 S-boxes operations, one for each byte
- In software S-Box is usually implemented as a lookup table



Input Byte = xy

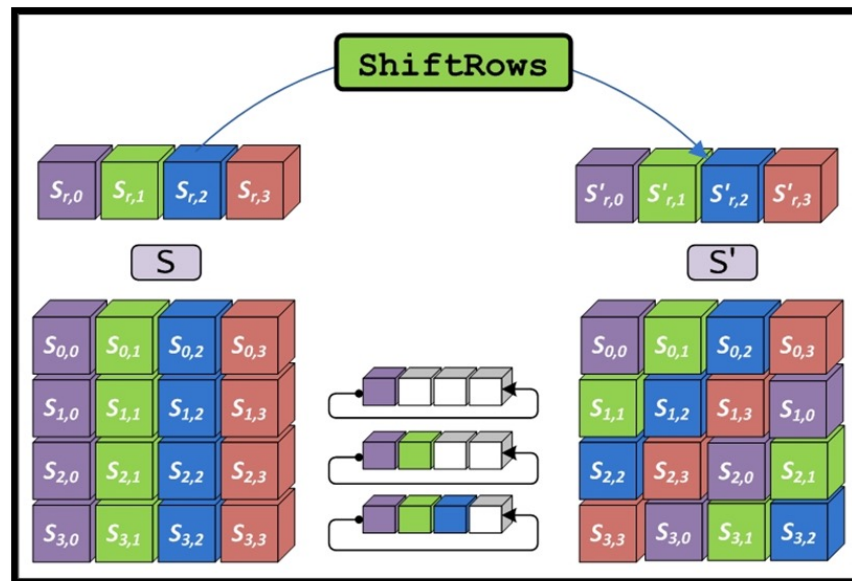
|   |   | y  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  |
| x | 0 | 63 | 7c | 77 | 7b | f2 | 6b | 6f | c5 | 30 | 01 | 67 | 2b | fe | d7 | ab | 76 |
|   | 1 | ca | 82 | c9 | 7d | fa | 59 | 47 | f0 | ad | d4 | a2 | af | 9c | a4 | 72 | c0 |
|   | 2 | b7 | fd | 93 | 26 | 36 | 3f | f7 | cc | 34 | a5 | e5 | f1 | 71 | d8 | 31 | 15 |
|   | 3 | 04 | c7 | 23 | c3 | 18 | 96 | 05 | 9a | 07 | 12 | 80 | e2 | eb | 27 | b2 | 75 |
|   | 4 | 09 | 83 | 2c | 1a | 1b | 6e | 5a | a0 | 52 | 3b | d6 | b3 | 29 | e3 | 2f | 84 |
|   | 5 | 53 | d1 | 00 | ed | 20 | fc | b1 | 5b | 6a | cb | be | 39 | 4a | 4c | 58 | cf |
|   | 6 | d0 | ef | aa | fb | 43 | 4d | 33 | 85 | 45 | f9 | 02 | 7f | 50 | 3c | 9f | a8 |
|   | 7 | 51 | a3 | 40 | 8f | 92 | 9d | 38 | f5 | bc | b6 | da | 21 | 10 | ff | f3 | d2 |
|   | 8 | cd | 0c | 13 | ec | 5f | 97 | 44 | 17 | c4 | a7 | 7e | 3d | 64 | 5d | 19 | 73 |
|   | 9 | 60 | 81 | 4f | dc | 22 | 2a | 90 | 88 | 46 | ee | b8 | 14 | de | 5e | 0b | db |
|   | a | e0 | 32 | 3a | 0a | 49 | 06 | 24 | 5c | c2 | d3 | ac | 62 | 91 | 95 | e4 | 79 |
|   | b | e7 | c8 | 37 | 6d | 8d | d5 | 4e | a9 | 6c | 56 | f4 | ea | 65 | 7a | ae | 08 |
|   | c | ba | 78 | 25 | 2e | 1c | a6 | b4 | c6 | e8 | dd | 74 | 1f | 4b | bd | 8b | 8a |
|   | d | 70 | 3e | b5 | 66 | 48 | 03 | f6 | 0e | 61 | 35 | 57 | b9 | 86 | c1 | 1d | 9e |
|   | e | e1 | f8 | 98 | 11 | 69 | d9 | 8e | 94 | 9b | 1e | 87 | e9 | ce | 55 | 28 | df |
|   | f | 8c | a1 | 89 | 0d | bf | e6 | 42 | 68 | 41 | 99 | 2d | 0f | b0 | 54 | bb | 16 |

# Advanced Encryption Standard – ShiftRows



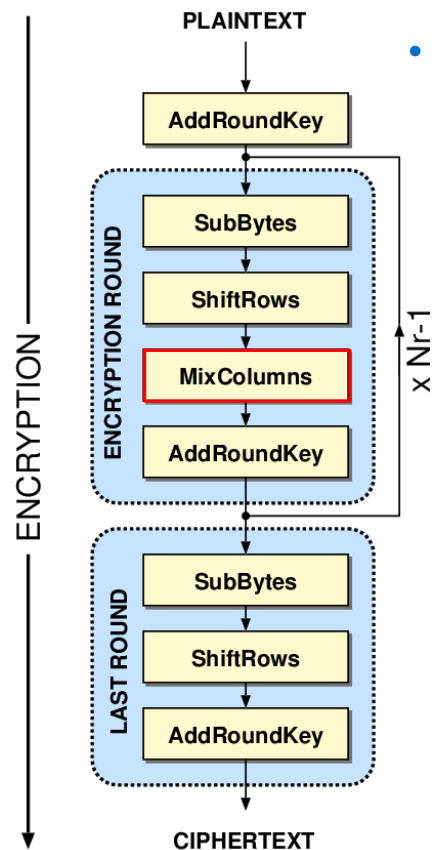
- **ShiftRows**

- Performs permutations on a byte level (rounded shift)
- Permutation consists of either 0, 1, 2, or 3 row-wise left rotational shifts in the state matrix





# Advanced Encryption Standard – MixColumns

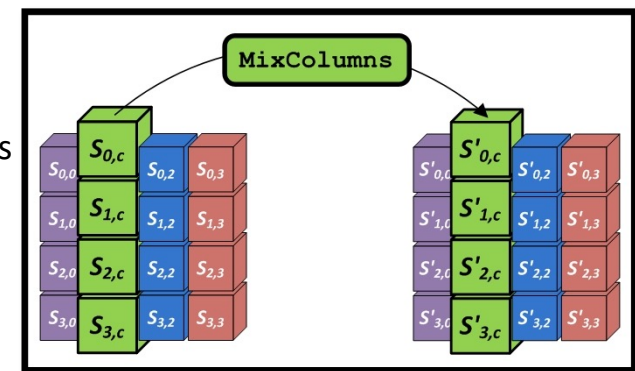


## • MixColumns

- Update state matrix columns using a matrix vector multiplication
- Matrix values are predefined:

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot \begin{pmatrix} S_{0,c} \\ S_{1,c} \\ S_{2,c} \\ S_{3,c} \end{pmatrix} = \begin{pmatrix} S'_{0,c} \\ S'_{1,c} \\ S'_{2,c} \\ S'_{3,c} \end{pmatrix}$$

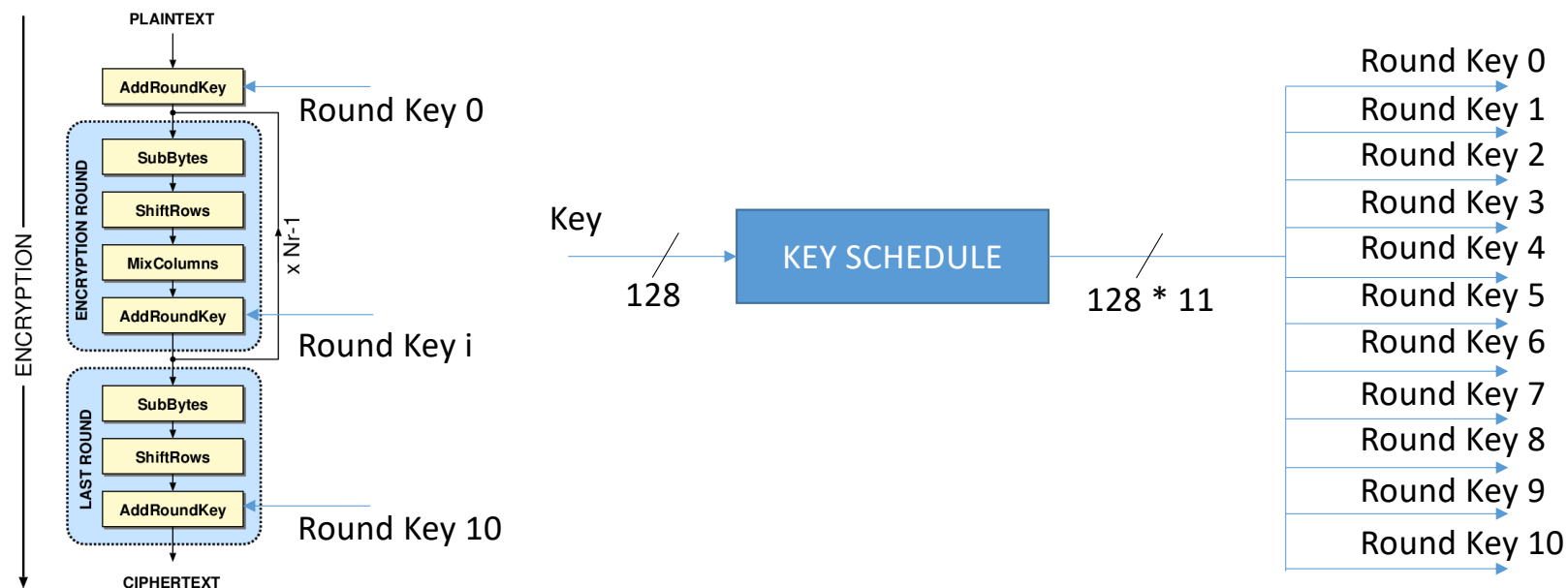
- These are not traditional multiplications and additions
  - Multiply by 01 – It is the value itself
  - Multiply by 02 – Left shift by 1 followed by a potential XOR operation. If the MSB input bit is 1 an extra XOR with “0001 1011” should be performed.
  - 03 - multiple by 2 and add, where add is an XOR operation
  - All other additions are XOR



# Advanced Encryption Standard – Key Schedule

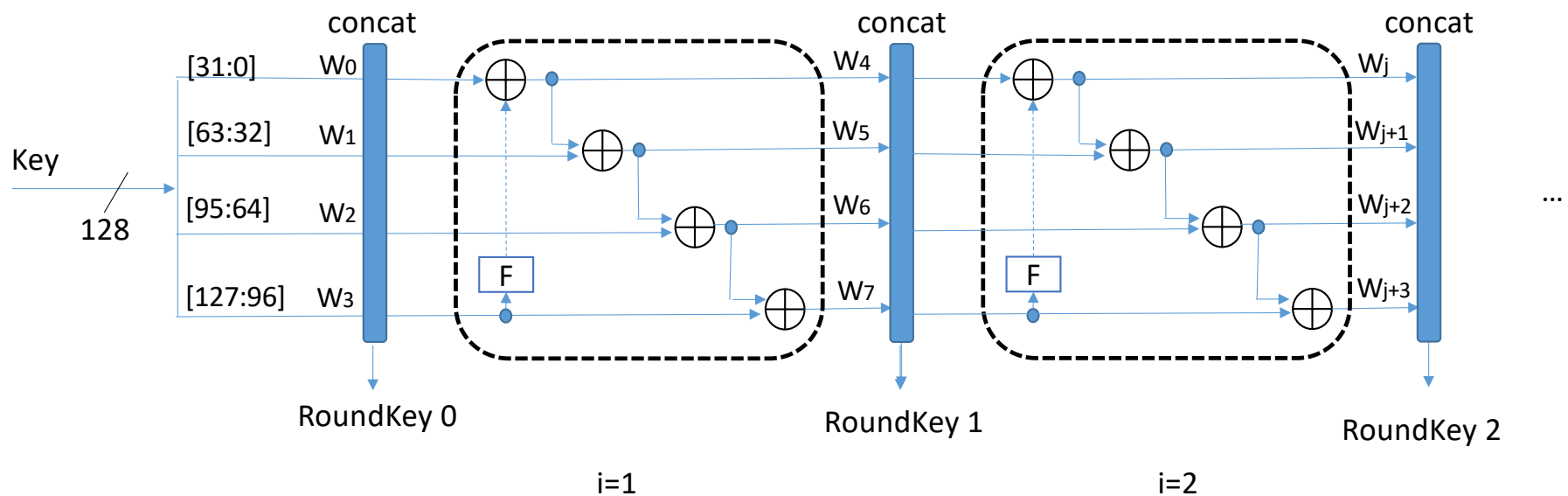
- Key Schedule

- RoundKeys are derived iteratively from the original key of 128/192/256 bits
- There are 11 Roundkeys for AES-128 (the first AddRoundKey + one per round)
- It is word-oriented: 32 bits



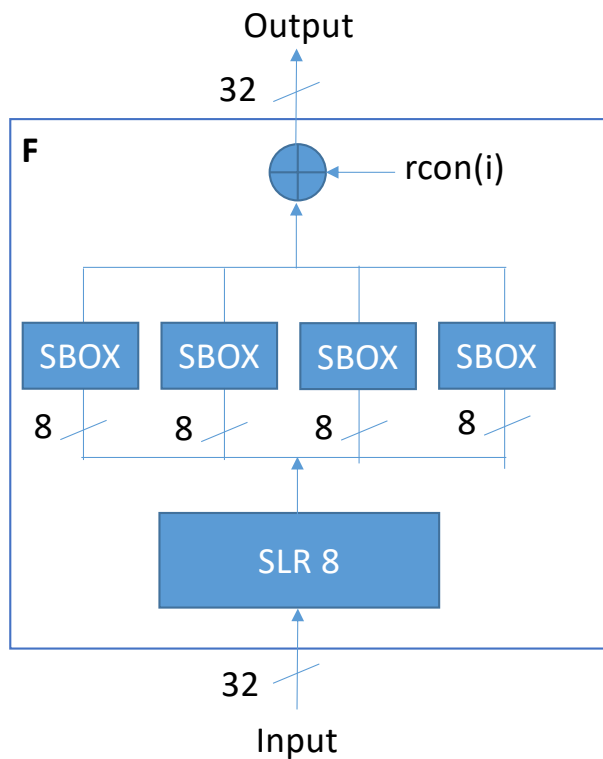
# Advanced Encryption Standard – Key Schedule

- Key Schedule Operation for AES128
  - Iterate over  $i$  rounds (10 rounds for AES128)



# Advanced Encryption Standard – Key Schedule

- Key Schedule Operation
  - F Function:



32-bit constant rcon

| $i$    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|--------|----|----|----|----|----|----|----|----|----|----|
| $rc_i$ | 01 | 02 | 04 | 08 | 10 | 20 | 40 | 80 | 1B | 36 |

# RISC-V – Scalar Cryptography Instruction Set Extension

## 2.5. Zkne - NIST Suite: AES Encryption

Instructions for accelerating the encryption and key-schedule functions of the AES block cipher.

| RV32 | RV64 | Mnemonic  | Instruction                                 |
|------|------|-----------|---------------------------------------------|
| ✓    |      | aes32esi  | AES final round encrypt (RV32)              |
| ✓    |      | aes32esmi | AES middle round encrypt (RV32)             |
|      | ✓    | aes64es   | AES encrypt final round instruction (RV64)  |
|      | ✓    | aes64esmi | AES encrypt middle round instruction (RV64) |
|      | ✓    | aes64ks1i | AES Key Schedule Instruction 1 (RV64)       |
|      | ✓    | aes64ks2  | AES Key Schedule Instruction 2 (RV64)       |

## 2.4. Zknd - NIST Suite: AES Decryption

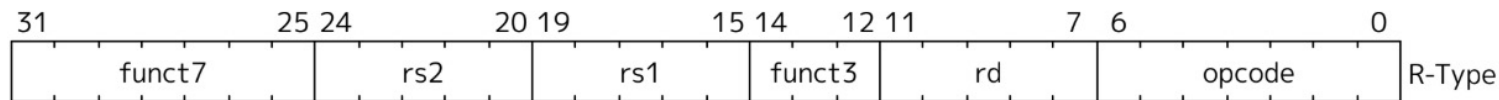
Instructions for accelerating the decryption and key-schedule functions of the AES block cipher.

| RV32 | RV64 | Mnemonic  | Instruction                               |
|------|------|-----------|-------------------------------------------|
| ✓    |      | aes32dsi  | AES final round decrypt (RV32)            |
| ✓    |      | aes32dsmi | AES middle round decrypt (RV32)           |
|      | ✓    | aes64ds   | AES decrypt final round (RV64)            |
|      | ✓    | aes64dsm  | AES decrypt middle round (RV64)           |
|      | ✓    | aes64im   | AES Decrypt KeySchedule MixColumns (RV64) |
|      | ✓    | aes64ks1i | AES Key Schedule Instruction 1 (RV64)     |
|      | ✓    | aes64ks2  | AES Key Schedule Instruction 2 (RV64)     |

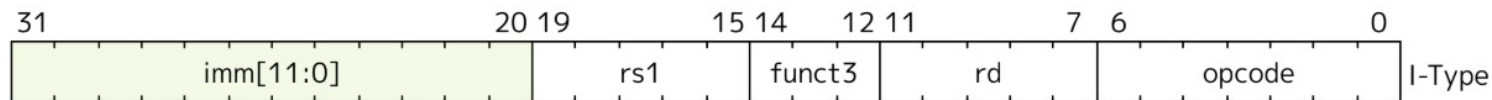
The aes32esmi and aes32dsmi instructions **only process one byte at a time** and **implement only a partial MixColumns step**. This is because a full MixColumns operation requires simultaneous processing of all four bytes in a column to perform the necessary finite field matrix multiplication, which is not feasible with byte-wise operations alone.

[2] RISC-V Cryptography Extensions Volume I: Scalar & Entropy Source Instructions, Version 0.9.3, 14th June 2021.  
Available at: <https://lists.riscv.org/g/dev-partners/attachment/43/0/riscv-crypto-spec-scalar-v0.9.3-DRAFT.pdf>

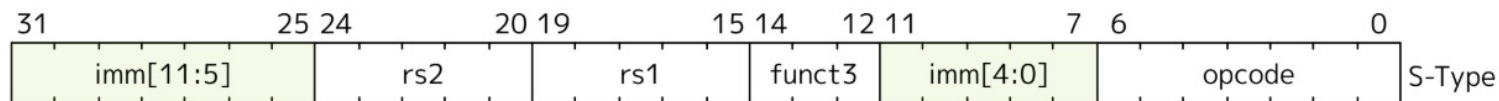
# RISC-V – ISA



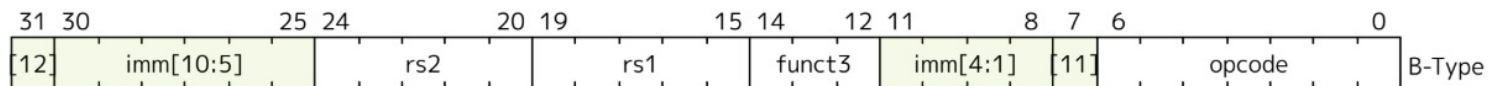
Operates on Registers



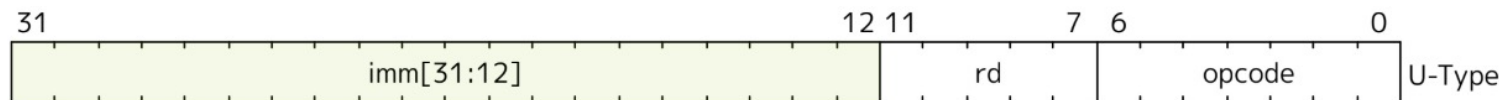
Uses Immediate constants



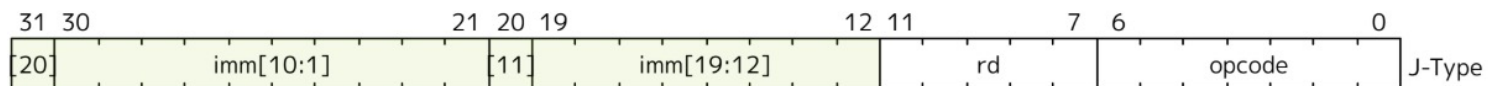
Stores values



Branches



Upper immediate (Big constants)



Jumps

[1] The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture, Version 20240411, Available at: <https://if-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications>

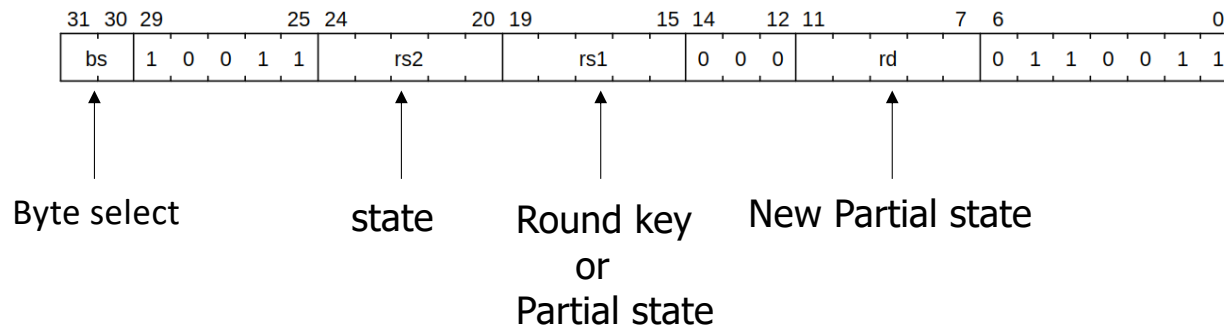
## AES - RISC-V Scalar Instructions

- Encryption – Middle round

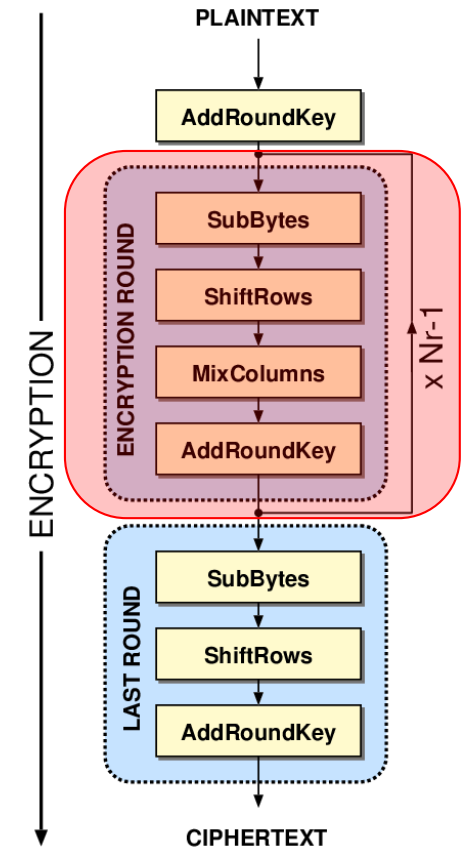
### Mnemonic

```
aes32esmi rd, rs1, rs2, bs
```

## Encoding



Takes a single byte from register rs2 according to bs (2 bit) and applies SBOX operation. Then, it applies a **partial MixColumns** operation. Finally, the intermediate result is rotated bs and XOR'ed with the value in register rs1.



# AES - RISC-V Scalar Instructions

- Decryption – Middle Round

## Mnemonic

aes32dsmi rd, rs1, rs2, bs

## Encoding

|    |    |    |    |   |   |    |     |  |  |     |    |  |   |    |    |    |   |  |   |   |   |   |   |   |   |
|----|----|----|----|---|---|----|-----|--|--|-----|----|--|---|----|----|----|---|--|---|---|---|---|---|---|---|
| 31 | 30 | 29 | 25 |   |   | 24 | 20  |  |  | 19  | 15 |  |   | 14 | 12 | 11 | 7 |  |   | 6 | 0 |   |   |   |   |
| bs |    | 1  | 0  | 1 | 1 | 1  | rs2 |  |  | rs1 |    |  | 0 | 0  | 0  | rd |   |  | 0 | 1 | 1 | 0 | 0 | 1 | 1 |

Takes a single byte from register rs2 according to bs (2 bit) and applies the **inverse** SBOX operation. Then, it applies an **inverse partial MixColumns** operation. Finally, the intermediate result is rotated bs and XOR'ed with the value in register rs1.



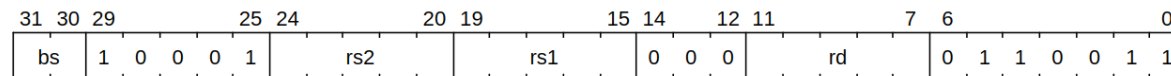
# AES - RISC-V Scalar Instructions

- Encryption – Final Round

## Mnemonic

aes32esi rd, rs1, rs2, bs

## Encoding



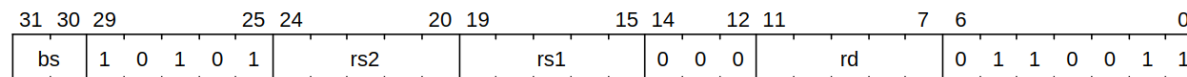
Takes a single byte from register rs2 according to bs (2 bit) and applies the SBOX operation. Finally, the intermediate result XOR'ed with the value in register rs1.

- Decryption – Final Round

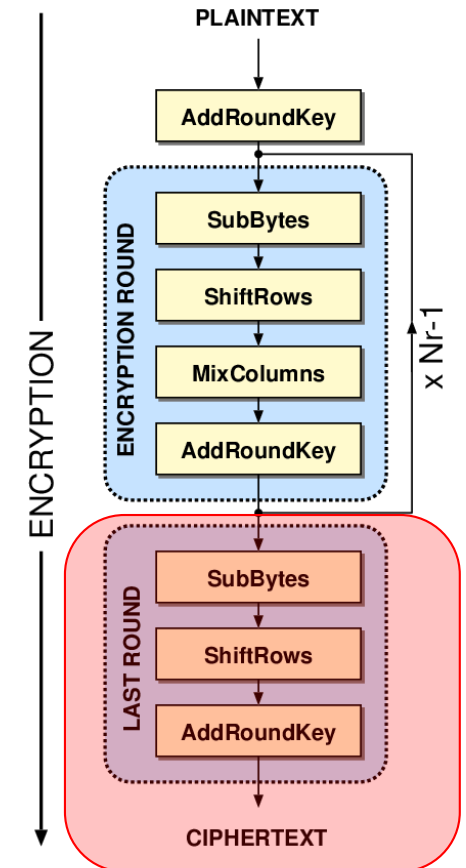
## Mnemonic

aes32dsi rd, rs1, rs2, bs

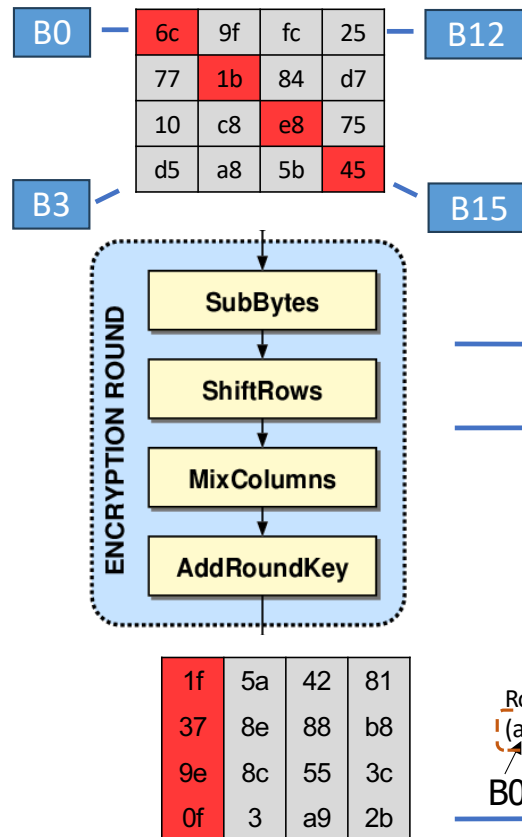
## Encoding



Takes a single byte from register rs2 according to bs (2 bit) and applies the inverse SBOX operation. Finally, the intermediate result XOR'ed with the value in register rs1.



# AES - Revisited



## Understanding how to use RISC-V AES scalar instructions

This example calculates the first output column of a middle round. Each value of a column in the output column depends on 4 values of the input (marked red).

|    |    |    |    |
|----|----|----|----|
| 50 | db | b0 | 3f |
| f5 | af | 5f | 0e |
| ca | e8 | 9b | 9d |
| 3  | c2 | 39 | 6e |

50 = SBOX(6c)  
af = SBOX(1b)  
9b = SBOX(e8)  
6e = SBOX(45)

|    |    |    |    |
|----|----|----|----|
| 50 | db | b0 | 3f |
| af | 5f | 0e | f5 |
| 9b | 9d | ca | e8 |
| 6e | 3  | c2 | 39 |

|    |    |    |    |
|----|----|----|----|
| bf | d2 | 61 | ab |
| cd | da | 2b | d4 |
| 60 | a0 | 6c | 4a |
| 18 | b2 | 90 | 2e |

bf = 2\*SBOX(6c) xor 3\*SBOX(1b) xor 1\*SBOX(e8) xor 1\*SBOX(45)  
cd = 1\*SBOX(6c) xor 2\*SBOX(1b) xor 3\*SBOX(e8) xor 1\*SBOX(45)  
60 = 1\*SBOX(6c) xor 1\*SBOX(1b) xor 2\*SBOX(e8) xor 3\*SBOX(45)  
18 = 3\*SBOX(6c) xor 1\*SBOX(1b) xor 1\*SBOX(e8) xor 2\*SBOX(45)

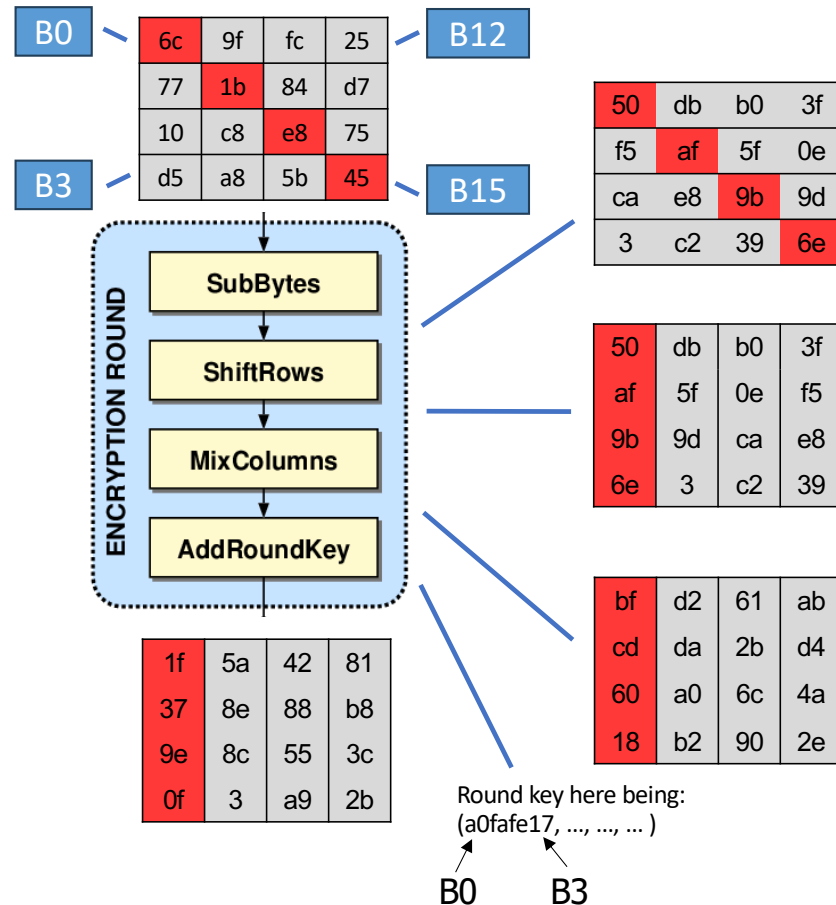
Round key here being:  
(a0fafe17, ..., ..., ...)

B0 B3

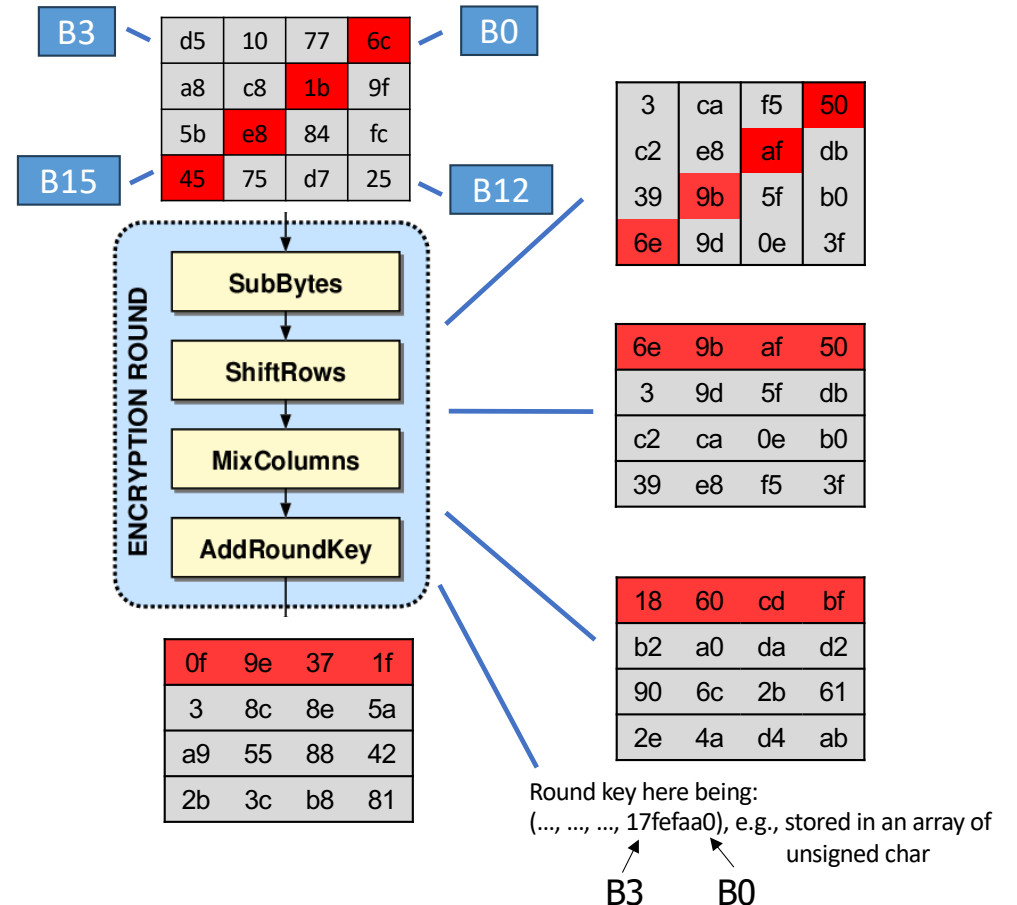
1f = 2\*SBOX(6c) xor 3\*SBOX(1b) xor 1\*SBOX(e8) xor 1\*SBOX(45) xor a0  
37 = 1\*SBOX(6c) xor 2\*SBOX(1b) xor 3\*SBOX(e8) xor 1\*SBOX(45) xor fa  
9e = 1\*SBOX(6c) xor 1\*SBOX(1b) xor 2\*SBOX(e8) xor 3\*SBOX(45) xor fe  
0f = 3\*SBOX(6c) xor 1\*SBOX(1b) xor 1\*SBOX(e8) xor 2\*SBOX(45) xor 17

# AES - Memory Layout

Column Based Indexing

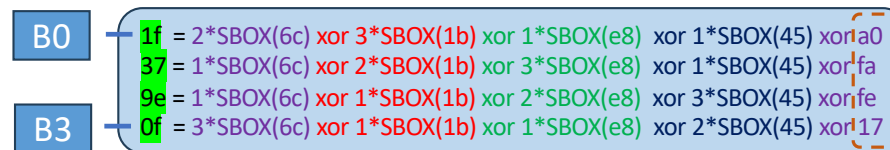


Row Based Indexing and MSB on left on a 32-bit proc.



# AES - RISC-V aes32esmi

- How to use these instructions to calculate an AES round?
  - Going back to the example where the first output column is calculated as:



Let...

- t0 initially hold the **first 4 bytes of the round key** -> 17fefaa0
- a4, a5, a6, and a7 (rs2) the **rows of the input** -> d510776c, a8c81b9f, 5be8f84c, 4575d725

Using four instructions it is possible to iteratively calculate 4 bytes of the middle round's output. For a full round this must be repeated four times, using different bytes of the input and the 3 other parts of the round key.

| aes32esmi t0,t0,a4,0                                                                           | aes32esmi t0,t0,a5, 1                                                                                                                                      | aes32esmi t0,t0,a6, 2                                                                                                                                                                                                  | aes32esmi t0,t0,a7,3                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0th byte: 6c                                                                                   | 1st byte: 1b                                                                                                                                               | 2nd byte: e8                                                                                                                                                                                                           | 3rd byte: 45                                                                                                                                                                                                                                                           |
| [3*SBOX(6c) @ SBOX(6c) @ SBOX(6c) @ 2*SBOX(6c)] rot 0                                          | [3*SBOX(1b) @ SBOX(1b) @ SBOX(1b) @ 2*SBOX(1b)] rot 1                                                                                                      | [3*SBOX(e8) @ SBOX(e8) @ SBOX(e8) @ 2*SBOX(e8)] rot 2                                                                                                                                                                  | [3*SBOX(45) @ SBOX(45) @ SBOX(45) @ 2*SBOX(45)] rot 3                                                                                                                                                                                                                  |
| t0 = {<br>3*SBOX(6c) xor 17,<br>1*SBOX(6c) xor fe,<br>1*SBOX(6c) xor fa,<br>2*SBOX(6c) xor a0} | t0 = {<br>1*SBOX(1b) xor 3*SBOX(6c) xor 17,<br>1*SBOX(1b) xor 1*SBOX(6c) xor fe,<br>2*SBOX(1b) xor 1*SBOX(6c) xor fa,<br>3*SBOX(1b) xor 2*SBOX(6c) xor a0} | t0 = {<br>1*SBOX(e8) xor 1*SBOX(1b) xor 3*SBOX(6c) xor 17,<br>2*SBOX(e8) xor 1*SBOX(1b) xor 1*SBOX(6c) xor fe,<br>3*SBOX(e8) xor 2*SBOX(1b) xor 1*SBOX(6c) xor fa,<br>1*SBOX(e8) xor 3*SBOX(1b) xor 2*SBOX(6c) xor a0} | t0 = {<br>2*SBOX(45) xor SBOX(e8) xor SBOX(1b) xor 3*SBOX(6c) xor 17,<br>3*SBOX(45) xor 2*SBOX(e8) xor SBOX(1b) xor SBOX(6c) xor fe,<br>1*SBOX(45) xor 3*SBOX(e8) xor 2*SBOX(1b) xor SBOX(6c) xor fa,<br>1*SBOX(45) xor SBOX(e8) xor 3*SBOX(1b) xor 2*SBOX(6c) xor a0} |
| t0 = e7aeaa00                                                                                  | t0 = 4801efea                                                                                                                                              | t0 = d32c5971                                                                                                                                                                                                          | t0 = 0f9e371f                                                                                                                                                                                                                                                          |

# AES - RISC-V aes32esi

- How to use these instructions to calculate the final AES round?

Let...

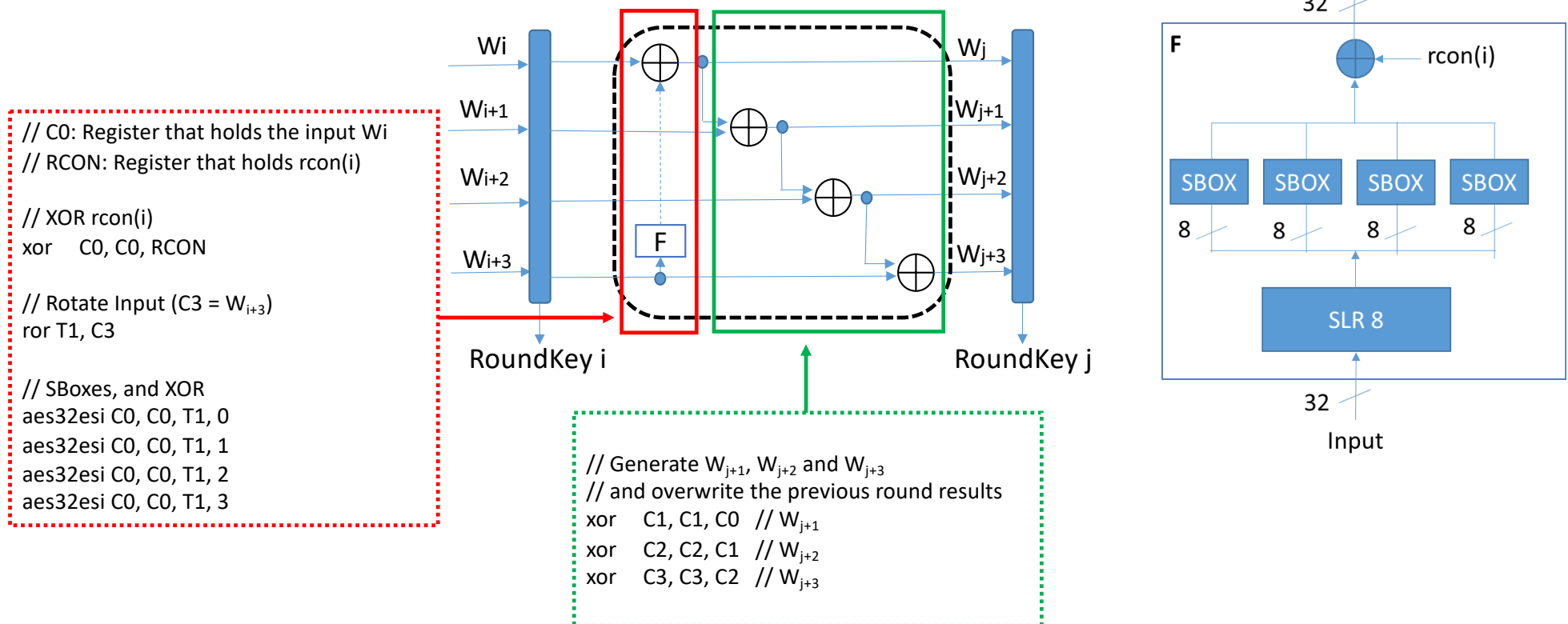
- t0 initially hold the **first 4 bytes of the round key** -> a8f914d0
- a4, a5, a6, and a7 (rs2) the **rows of the input** -> 2ea3ac24, 7dd15b67, 545e83d9, b3e2ab03

Using four instructions it is possible to iteratively calculate 4 bytes of the final round's output. For a full round this must be repeated four times, using different bytes of the input and the 3 other parts of the round key.

| aes32esi t0,t0,a4,0                                                         | aes32esi t0,t0,a5, 1                                                                                                | aes32esi t0,t0,a6, 2                                                                                                                                         | aes32esi t0,t0,a7,3                                                                                                                                                                                 |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0th byte: 2e                                                                | 1st byte: d1                                                                                                        | 2nd byte: 83                                                                                                                                                 | 3rd byte: 03                                                                                                                                                                                        |
| {SBOX(2e), 24'b0} rot 0                                                     | {SBOX(d1), 24'b0} rot 1                                                                                             | {SBOX(83), 24'b0} rot 2                                                                                                                                      | {SBOX(03), 24'b0} rot 3                                                                                                                                                                             |
| t0 = {<br>SBOX(2e) xor d0,<br>8'b0 xor 14,<br>8'b0 xor f9,<br>8'b0 xor a8 } | t0 = {<br>8'b0 xor SBOX(2e) xor d0,<br>SBOX(d1) xor 8'b0 xor 14,<br>8'b0 xor 8'b0 xor f9,<br>8'b0 xor 8'b0 xor a8 } | t0 = {<br>8'b0 xor 8'b0 xor SBOX(2e) xor d0,<br>8'b0 xor SBOX(d1) xor 8'b0 xor 14,<br>SBOX(83) xor 8'b0 xor 8'b0 xor f9,<br>8'b0 xor 8'b0 xor 8'b0 xor a8, } | t0 = {<br>8'b0 xor 8'b0 xor 8'b0 xor SBOX(2e) xor d0,<br>8'b0 xor 8'b0 xor SBOX(d1) xor 8'b0 xor 14,<br>8'b0 xor SBOX(83) xor 8'b0 xor 8'b0 xor f9,<br>SBOX(03) xor 8'b0 xor 8'b0 xor 8'b0 xor a8 } |
| t0 = ..                                                                     | t0 = ...                                                                                                            | t0 = ...                                                                                                                                                     | t0 = e12a15d3                                                                                                                                                                                       |

## AES - RISC-V Scalar Instructions

- How to use these instructions for the key scheduling (1/2)?
  - The final round instructions are used for the SBoxes and XOR'ing in the F function



# AES - RISC-V : That's ALL!

